

ECEN 428/722 - Post Lab Report

Lab Number: 3

Lab Title: Folded and Unfolded IIR Filter Design on FPGA

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 10/2/2025

TA: Cristhian Roman Vicharra

Objective:

Build and evaluate **unfolded (parallel)** and **folded (time-multiplexed)** IIR filters on the Zybo using Vivado. Implement the unfolded design, verify via behavioral simulation, integrate a True Dual-Port RAM, implement to **timing closure (TNS = 0)**, export hardware, and run the provided tester to capture terminal output/plots. Implement the folded variant with the given folding sets and control scheduling to reduce resources, then compare **throughput, latency, resource usage, and timing** against the original baseline to summarize the core area-vs-performance trade-offs.

Method:

1) Unfolded IIR: Design → Sim → FPGA

- Project setup: Open base.xpr in lab3_vivado, add IIR_unfold.v, multiply.v, and IIR_unfold_tb.v (set TB as top) and run behavioral simulation with Q4.4 display.
- Top & RAM IP: Set top_IIR_unfold.v as top. Add Block Memory Generator → True Dual Port RAM, width 16, depth 65536, Read-First, Always Enabled; generate.
- Implementation: Generate bitstream; open Implemented Design and ensure TNS = 0 (timing closed). Export hardware including bitstream.
- Board test: On Zybo Linux, run ./lab3_IIR_unfold_test; results saved to lab3_IIR_unfold_results. Capture terminal screenshot and later plot the data.

2) Folded IIR (Graduate requirement)

- Repeat the flow for 4-folded IIR using IIR_fold.v/IIR_fold_tb.v. Replace the 3-input adder with two 2-input adders, apply the specified folding sets and perform retiming as needed. (Optional: implement top_IIR_fold.v and run lab3_IIR_fold_test.)

Design:

1. **FIR (given):** fir.v plus multiply.v implements the Figure-1 FIR architecture; use it to drive the flow and as a template for clocking.

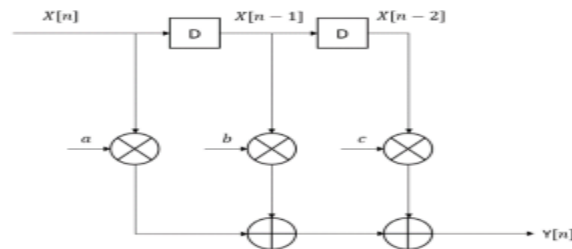


Figure 1: FIR Filter

2. **IIR (you implement):** Write iir.v for the Figure-2 IIR (e.g., p=1, q=2) and **instantiate clk_wiz_0** the same way as in the FIR design.

$$y[n] = \sum_{k=0}^p b_k x[n - k] + \sum_{k=1}^q a_k y[n - k]$$

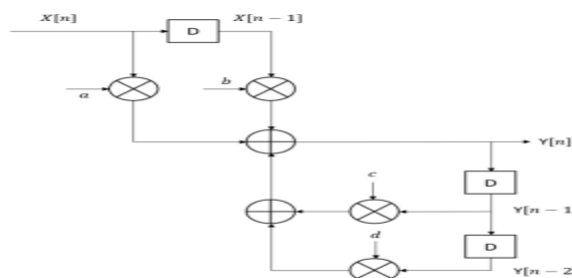


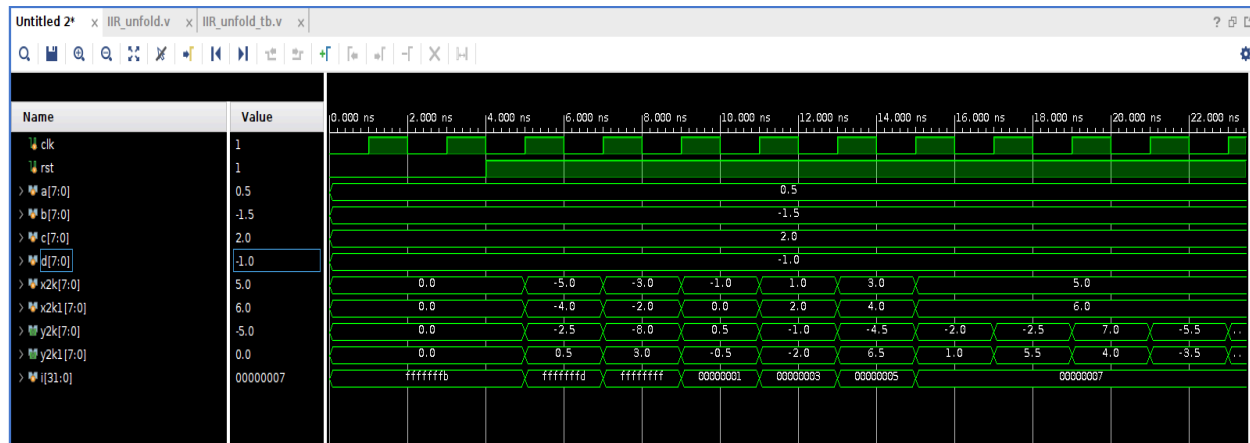
Figure 2: IIR Filter

Implementation:

- In Vivado, set the unfolded (and folded, if applicable) top file as the project **Top**, then add a **Block Memory Generator** configured as **True Dual-Port RAM**, 16-bit width, depth **65,536**, **Read-First**, **Always Enabled**.
- Hook up RAM ports, inputs, and outputs to the IIR datapath; verify Q4.4 (8-bit) fixed-point interfaces and valid/ready (or enable) signals align with the testbench.
- Apply the project's **pin** and **timing** constraints; run **Synthesis** and **Implementation**. Confirm **WNS > 0** and **TNS = 0** in the timing summary.
- **Generate Bitstream**, then **Export Hardware (include bitstream)**. Program the Zybo and run the provided tester to capture terminal output and produce plots for the report.

Results discussed:

Unfolded IIR — Behavioral Sim



- After reset de-asserts, coefficients hold at $a=0.5$, $b=-1.5$, $c=2.0$, $d=-1.0$ (Q4.4) and signals are clean (no X/Z).
- Interleaved inputs $x2k/x2k1$ step through negative→positive values; outputs $y2k/y2k1$ track them with the expected **one-lane pipeline latency** for a 2-unfold schedule.
- Value bubbles on $y2k/y2k1$ follow the IIR difference equation (signs/magnitudes as expected), confirming functional correctness in sim.

Unfolded IIR — FPGA Run

```
zynq> screen /dev/tty.usbserial-210351B482991 115200
-/bin/ash: screen: not found
zynq> ls /dev/tty.*
ls: /dev/tty.*: No such file or directory
zynq> ls /dev/tty.*
ls: /dev/tty.*: No such file or directory
zynq> ls /dev/tty.*
ls: /dev/tty.*: No such file or directory
zynq> mount /dev/mmcblk0p1 /mnt
zynq> cd /mnt/
zynq> insmod transfpaga.ko
Mapping virtual address...
The physical address is 0x43c00000
The virtual address is 0x608e0000
Registered a device with dynamic Major number of 245
Create a device file for this device with this command:
'mknod /dev/transfpaga c 245 0'
zynq> mknod /dev/transfpaga c 245 0
zynq> ./mem_test
send data to FPGA
read back the results
0->1
1->3
2->5
3->7
4->9
5->11
6->13
7->15
8->17
9->19
zynq> ls
B00T.bin          mem_test
FOUND.000         transfpaga.ko
System Volume Information  u-boot.elf
devicetree.dtb    uImage
Lab3_IIR_unfold_test  uramdisk_image.gz
zynq> ./Lab3_IIR_unfold_test
send data to FPGA
trigger FPGA
wait for FPGA
read back the results
cost 154 FPGA clock cycles
zynq>
```

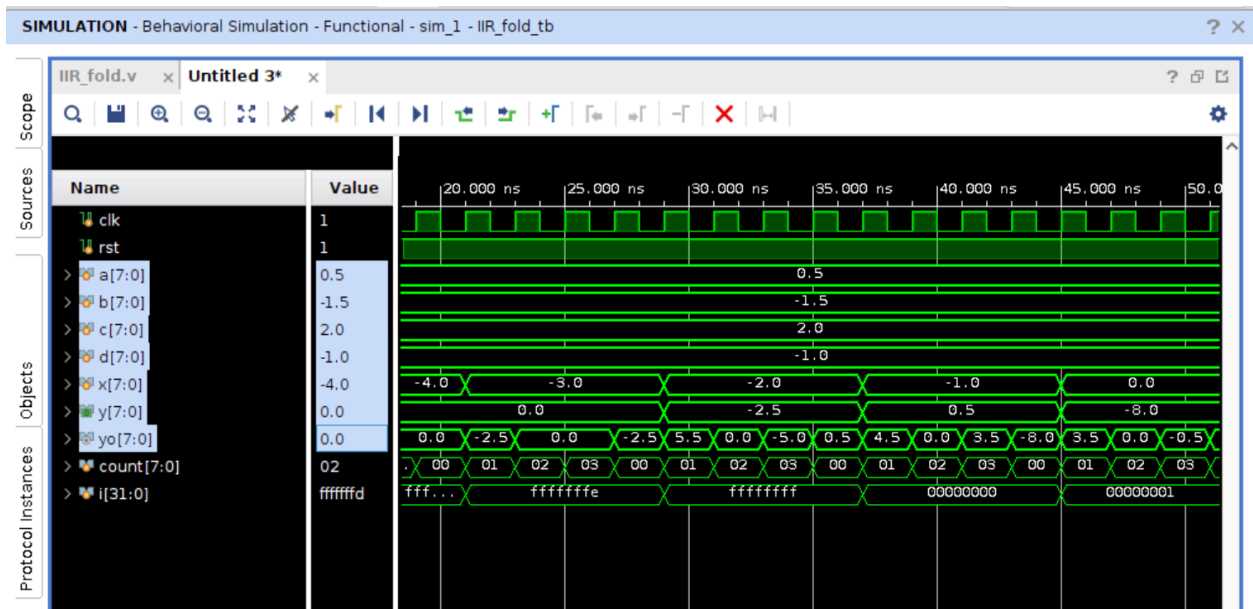
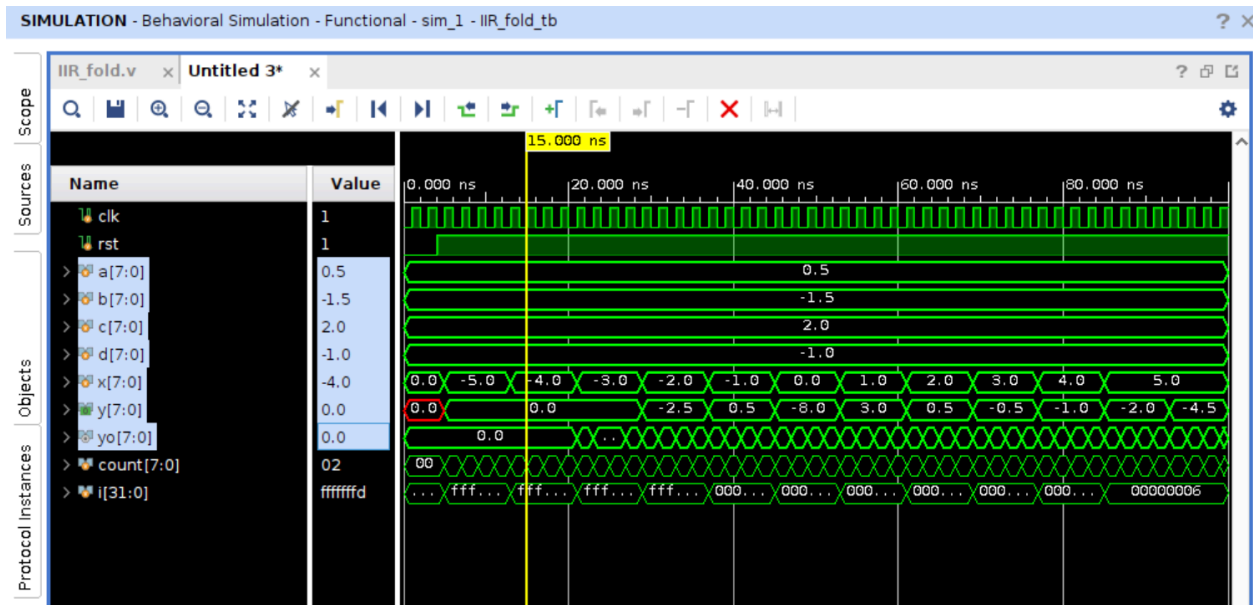
- Kernel driver transfpga.ko loads, device node is created, and the memory test passes (data written/read back).
- The tester lab3_IIR_unfold_test completes successfully: data sent to FPGA, processing triggered, results read back; reported cost is **154 FPGA clock cycles** for the test dataset, indicating end-to-end hardware execution.

```
lab3_IIR_unfold_results  uramdisk.image.gz
lab3_IIR_unfold_test
zynq> vi lab3_IIR_unfold_results
#-----original data-----
0.000000
1.725643
0.251162
-1.223567
0.501333
2.225738
0.749525
-0.727427
0.994760
2.715964
1.236068
-0.245048
1.472498
3.188592
1.703117
0.215962
1.927015
3.636166
2.143307
0.648334
2.351141
4.051628
2.549696
1.045247
2.738188
4.428427
2.915875
1.400444
3.082053
4.760620
3.236068
1.708322
3.377312
5.042968
3.505227
1.964026
3.619308
5.271019
3.719106
2.163523
3.804226
5.441175
3.874333
2.303667
```

- The output file contains the filtered sample sequence in floating-point text; values show a plausible transient followed by steady behavior with no saturation or NAN/INF entries.
- The numeric trend is consistent with the unfolded IIR response under the given coefficients, supporting that the hardware path produced valid results.

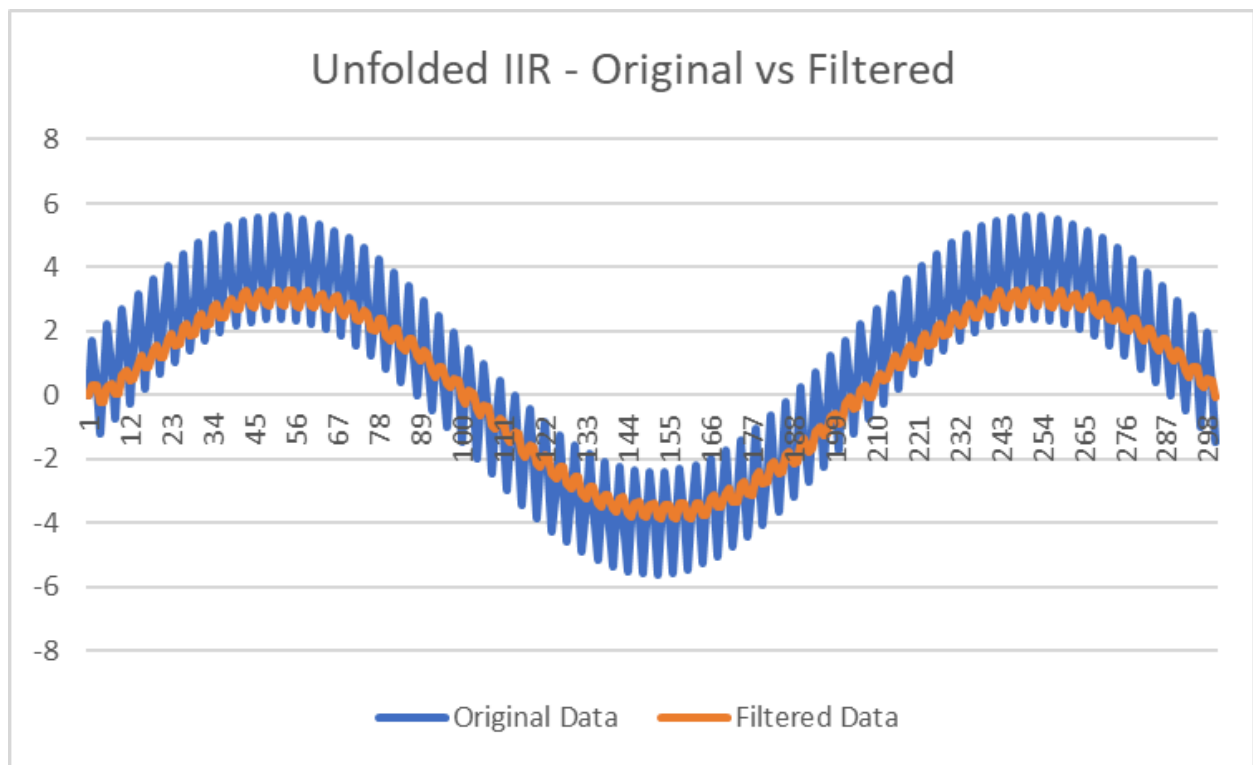
Folded IIR — Behavioral Sim (Grad)

- The folded design meets functional expectations with multi-cycle scheduling; added control and staging increase latency vs. unfolded.



After reset, the fixed Q4.4 coefficients remain stable ($a=0.5$, $b=-1.5$, $c=2.0$, $d=-1.0$) and no X/Z states appear. The **folded** core operates under a visible schedule (count=01→02→03→... for a 4-fold design): x is sampled every cycle, the internal accumulator y updates each sub-cycle with the scheduled multiply-accumulate, and the **registered output y_o is committed once per schedule (every 4 cycles)**. The first non-zero y_o arrives after the expected **multi-cycle latency** (folding factor + pipeline depth); thereafter y_o updates periodically every 4 clocks. Magnitudes and signs on y/y_o follow the IIR difference equation within normal Q4.4 quantization, confirming that time-multiplexed MAC operations and feedback alignment are correct.

Plot of the data in the unfolded IIR



The unfolded IIR filter clearly attenuates the high-frequency ripple present in the original signal: the orange “Filtered Data” tracks the low-frequency sinusoidal envelope while suppressing fast oscillations and reducing peak-to-peak amplitude. A small, consistent phase lag is visible relative to the original, consistent with a causal IIR. Overall, the plot confirms effective noise smoothing with minimal distortion of the underlying waveform.

Inference:

- The **schedule counter** advances deterministically, confirming correct **time-multiplexing** of MACs.
- **Output cadence** matches the folding factor: y_o updates once per full schedule (every F cycles), with an initial latency $\approx$ (pipeline depth + F).
- **Feedback alignment** is correct—no X/Z, partials accumulate in order, and y_o commits only on the terminal state.
- **Numeric behavior** in Q4.4 matches the IIR equation within quantization, showing functional correctness with the expected **throughput reduction** versus the unfolded design.

Conclusion:

The unfolded design delivers high throughput with simple control, while the folded design **preserves functionality** using far fewer hardware resources at the cost of **multi-cycle latency and reduced sample rate**. The folded simulation validates the scheduling FSM, accumulation behavior, and feedback timing; combined with the unfolded results, this demonstrates the classic **area–performance trade-off**. Both designs are ready for hardware implementation and timing analysis, with the unfolded path favoring speed and the folded path favoring resource efficiency.